

Batch Normalization using an SVD Layer

KAUST Academy Summer School 2026 — Weekly Project Report 2

Abdulmohsen Mohammedsaleh, Majed Alsulami, Yousef Alogiley,
Jad Mounayer

abdulmohsen1284@gmail.com, majedsaadalsulami@gmail.com, yousef.alogiley@gmail.com,
Jad.mounayer@outlook.com

Abstract

In the second project week we acted on our mentor's feedback and made our Week-1 comparison of Batch Normalization (BN) and the SVD-based Rank Reduction (RR) layer statistically sound. We fixed the previously unseeded data generation, re-implemented BN from scratch (no `nn.BatchNorm1d`), replaced the custom-gradient SVD with a simpler `torch.linalg.svd`-based `stable_svd`, and repeated every experiment over five random seeds, reporting mean $\pm$ standard deviation. We further ran two ablations: shrinking the feature extractor (3, 2, and 1 convolutional blocks) and shrinking the training set (800 down to 150 samples). Across all settings the ranking is unchanged and now statistically robust: the plain network localizes best (test ℓ_2 error $1.8 \times 10^{-3} \pm 3 \times 10^{-4}$), the RR bottleneck degrades it mildly ($8.0 \times 10^{-3} \pm 1.3 \times 10^{-3}$), and the BN bottleneck degrades it an order of magnitude ($4.5 \times 10^{-2} \pm 1.3 \times 10^{-2}$) — a gap that *widens* as the extractor gets shallower or data get scarcer, while the RR layer degrades gracefully. Swapping the custom SVD backward for native autograd changed results only marginally, indicating the custom gradient was not a source of instability on this task.

📅 **Date:** July 21, 2026

Contents

1 Introduction 2

2 Additional Related Work 2

3 Methodology 2

3.1 Fixed data generation 3

3.2 Manual Batch Normalization 3

3.3 Simplified stable SVD 3

3.4 Experimental protocol 4

4 Results 4

4.1 Multi-seed comparison 4

4.2 Custom SVD backward vs. native autograd 4

4.3 Architecture ablation 5

4.4 Data reduction 5

5 Discussion 7

6 Limitations and Future Work 7

7 Conclusion 8

References 9

1 Introduction

Week 1 introduced our study object: the Rank Reduction (RR) layer of Rank Reduction Autoencoders [1], which reconstructs each mini-batch of activations from its top- k singular directions — the optimal rank- k approximation by the Eckart–Young theorem [2] — and its comparison against Batch Normalization (BN) [3] in the same architectural slot. Our first experiment (one run, one seed) found the plain network (OA) best, the RR variant (OA_RR) second, and the BN variant (OA_BN) last on a synthetic Gaussian-blob localization task.

That conclusion rested on a single run of a pipeline whose data generation was not even seeded. This week we address the mentor’s feedback point by point:

- 1. Reading.** We broadened the literature review to other normalization schemes (LayerNorm, GroupNorm) and to work analyzing *why* BN helps optimization (Section 2).
- 2. Manual BN.** We implemented Batch Normalization from scratch rather than using the built-in layer (Section 3.2).
- 3. Fixed data generation and multiple seeds.** Data generation is now driven by a dedicated seeded RNG, and every experiment is repeated over five seeds with mean $\pm$ standard deviation reported (Section 3.4).
- 4. SVD stability.** We replaced the custom-backward SVD with a plain `torch.linalg.svd`-based `stable_svd` and compared the two head-to-head (Section 3.3).
- 5. Ablations.** We varied the feature-extractor depth (3, 2, 1 convolutional blocks) and the training-set size (800, 500, 300, 150 samples) to test whether the Week-1 ranking is an artifact of one architecture or of abundant data (Sections 4.3, 4.4).

2 Additional Related Work

Normalization beyond BN.. Batch Normalization [3] standardizes each feature using *batch* statistics, which makes its training-time behavior depend on batch composition and size. Layer Normalization [4] instead normalizes across the features of each *single* sample, removing the batch dependence entirely (and enabling use in RNNs and Transformers); Group Normalization [5] normalizes over channel groups per sample and matches BN at small batch sizes where BN’s statistics become noisy. These works matter for our project because the RR layer, like BN, is a *batch-statistics* method: its per-batch SVD basis is exactly the kind of batch-dependent quantity that LayerNorm and GroupNorm were designed to avoid, and its train/inference discrepancy (per-batch basis vs. finalized basis) mirrors BN’s batch-statistics vs. running-statistics discrepancy.

Why does BN help?. BN was motivated by reducing *internal covariate shift*, but Santurkar et al. [6] show empirically and theoretically that its main effect is smoothing the optimization landscape (better-conditioned loss and gradients), while Bjorck et al. [7] attribute its benefit largely to enabling larger learning rates, and Luo et al. [8] characterize its implicit regularization. A common thread is that BN’s benefits are *optimization* benefits that shine in deep networks trained at aggressive learning rates — conditions our small regression network does not meet, which helps explain the results in Section 4.

3 Methodology

3.1 Fixed data generation

Week 1's script seeded the model weights but *not* the data: every run of the notebook produced a different set of Gaussian-blob centers. All data are now generated by a dedicated numpy RNG with a fixed data seed, independent of the training seed. In addition, the full 800-sample training pool is generated once and smaller training sets are taken as *nested prefixes* of it, so the data-reduction experiment in Section 4.4 compares subsets of the same data rather than freshly resampled datasets. The task itself is unchanged: 64×64 images of an isotropic Gaussian blob ($\sigma = 0.2$) with center drawn uniformly from $[-0.5, 0.5]^2$, the target being the center coordinates; 200 held-out test samples.

3.2 Manual Batch Normalization

Following the mentor's request, OA_BN no longer uses `nn.BatchNorm1d`; we implement BN ourselves, directly from Eq. (1): batch statistics with learnable scale/shift during training, running statistics at inference.

$$\hat{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \quad y_j^{(i)} = \gamma_j \hat{x}_j^{(i)} + \beta_j. \quad (1)$$

```

1 class ManualBatchNorm1d(nn.Module):
2     def __init__(self, num_features, eps=1e-5, momentum=0.1):
3         super().__init__()
4         self.eps = eps; self.momentum = momentum
5         self.gamma = nn.Parameter(torch.ones(num_features)) # scale
6         self.beta = nn.Parameter(torch.zeros(num_features)) # shift
7         self.register_buffer("running_mean", torch.zeros(num_features))
8         self.register_buffer("running_var", torch.ones(num_features))
9
10    def forward(self, x):
11        if self.training:
12            mean = x.mean(dim=0)
13            var = x.var(dim=0, unbiased=False) # biased var for normalization
14            with torch.no_grad(): # running stats for inference
15                self.running_mean.mul_(1 - self.momentum).add_(self.momentum *
16                    mean)
17                self.running_var.mul_(1 - self.momentum).add_(
18                    self.momentum * x.var(dim=0, unbiased=True))
19            x_hat = (x - mean) / torch.sqrt(var + self.eps)
20        else:
21            x_hat = (x - self.running_mean) / torch.sqrt(self.running_var + self.
22                eps)
23        return self.gamma * x_hat + self.beta

```

We verified numerical agreement with `nn.BatchNorm1d` in both training and evaluation modes before using it in the experiments.

3.3 Simplified stable SVD

Week 1's RR layer used a hand-written `autograd.Function` implementing the SVD gradient, whose formula contains factors $1/(\sigma_i - \sigma_j)$ that are delicate when singular values nearly coincide [9]. To test whether that custom gradient causes problems, we adopted the mentor's simpler

replacement, which relies on the *native* autograd of `torch.linalg.svd` and only handles solver failures:

```

1 def stable_svd(X, full_matrices=False):
2     # cusolver's default gesvdj can fail to converge on ill-conditioned /
3     # repeated-singular-value matrices (common with grid-structured data).
4     # Use the accurate gesvd driver on CUDA, and fall back to CPU LAPACK
5     # if it still fails. Gradients flow through both paths.
6     try:
7         if X.is_cuda:
8             return torch.linalg.svd(X, full_matrices=full_matrices, driver='gesvd')
9         return torch.linalg.svd(X, full_matrices=full_matrices)
10    except torch.linalg.LinAlgError:
11        U, S, Vh = torch.linalg.svd(X.cpu(), full_matrices=full_matrices)
12        return U.to(X.device), S.to(X.device), Vh.to(X.device)

```

All Week-2 RR results use this `stable_svd`; Section 4.2 compares it against the old custom-backward version under identical seeds.

3.4 Experimental protocol

Models. As in Week 1, the three variants OA, OA_BN, and OA_RR share the same convolutional feature extractor and regression head and differ only in the bottleneck between them: identity, manual BN, or an RR layer with rank $k = 8$. For the depth ablation the extractor uses $d \in \{3, 2, 1\}$ conv-ReLU-maxpool blocks (channels 8, 16, 32), giving flattened feature dimensions $M = 2048, 4096$, and 8192 respectively.

Training. 20 epochs, Adam [10] ($\text{lr } 10^{-3}$), MSE loss, batch size 32 (`drop_last`); PyTorch [11]. For each seed $s \in \{0, \dots, 4\}$, `torch.manual_seed(s)` is called before constructing each variant, so all three start from identical convolutional and linear weights and see identical batch orders. At evaluation, OA_RR finalizes its inference basis from its basis history and OA_BN switches to running statistics.

Metrics. We report the mean Euclidean (ℓ_2) localization error on the 200 test samples, aggregated over the five seeds as $\text{mean} \pm \text{standard deviation}$. For the data-reduction study we additionally record the train-set ℓ_2 error (in evaluation mode) to expose overfitting via the train-test gap.

4 Results

4.1 Multi-seed comparison

Figure 1 and Table 1 show the base configuration (3 conv blocks, $N_{tr} = 800$) over five seeds. The Week-1 ranking survives with non-overlapping error bars: OA is best, OA_RR is $\sim 4.4\times$ worse, and OA_BN is a further $\sim 5.6\times$ worse ($25\times$ the OA error). Seed-to-seed variation is small for OA and OA_RR (relative std $\sim 11\text{--}16\%$) but noticeably larger for OA_BN ($\sim 27\%$).

4.2 Custom SVD backward vs. native autograd

Table 2 compares OA_RR trained with the Week-1 custom-backward SVD against the new `stable_svd` (native autograd), on identical seeds. Neither version produced a solver failure or a NaN in any of the 10 runs, so the custom gradient is *not* a source of hard instability on this task; the native version is nonetheless slightly more accurate and less variable, and is simpler, so we adopt it going forward.

Table 1 Base configuration over 5 seeds (mean $\pm$ std). Baseline: OA.

Model	Test mean ℓ_2	Test MSE
OA (baseline)	0.0018 ± 0.0002	$(2.2 \pm 0.5) \times 10^{-6}$
OA_BN	0.0445 ± 0.0119	$(1.4 \pm 0.7) \times 10^{-3}$
OA_RR	0.0080 ± 0.0013	$(5.2 \pm 1.5) \times 10^{-5}$

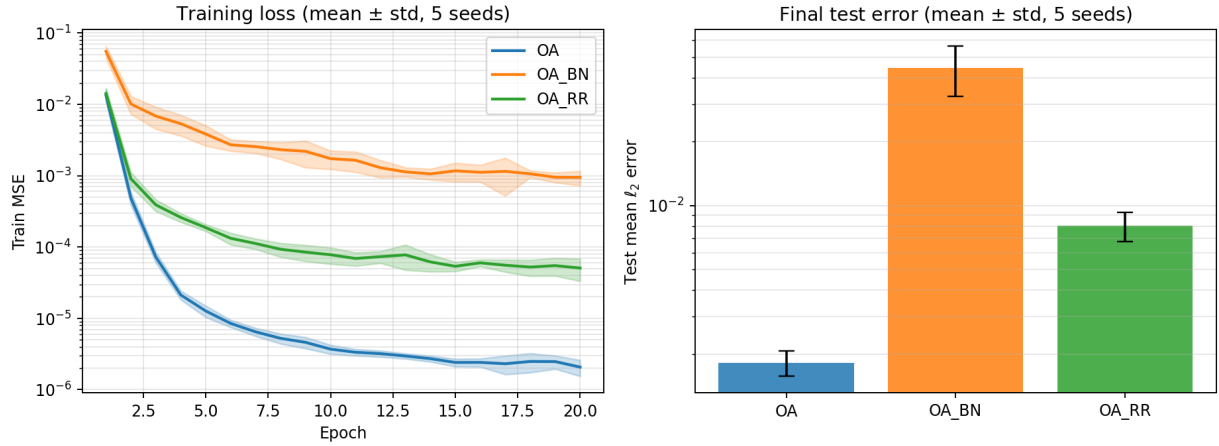


Figure 1 Base configuration over 5 seeds. Left: training-loss curves (mean $\pm$ std band). Right: final test ℓ_2 error (log scale).

4.3 Architecture ablation

Figure 2 varies the feature-extractor depth. The ranking $OA < OA_RR < OA_BN$ holds at every depth, so it is not an artifact of one architecture. The gaps *widen* as the extractor shrinks: with a single conv block, OA degrades mildly (0.0051 ± 0.0013), OA_RR degrades to 0.0375 ± 0.0070 , and OA_BN collapses to 0.17 ± 0.11 — with huge seed variance (individual seeds range from 0.07 to 0.36).

4.4 Data reduction

Figure 3 shrinks the training set from 800 to 150 samples (nested subsets of the same pool). All three models degrade smoothly and the ranking is again preserved at every size. The right panel shows the test-to-train error ratio: OA's ratio grows from ~ 1.1 at $N_{tr} = 800$ to ~ 1.7 at $N_{tr} = 150$ — overfitting is only *beginning* at 150 samples — while OA_BN's ratio stays near 1 because its error is dominated by the bottleneck itself, on train and test alike. Neither bottleneck overtakes the plain network at any size we tested: in this regime the errors they introduce far exceed the (still small) overfitting they might prevent.

Table 2 OA_RR with the two SVD implementations (5 seeds each).

SVD implementation	Test mean ℓ_2	Failures	NaNs
Custom backward (Week 1)	0.0108 ± 0.0027	0/5	0/5
stable_svd (native)	0.0080 ± 0.0013	0/5	0/5

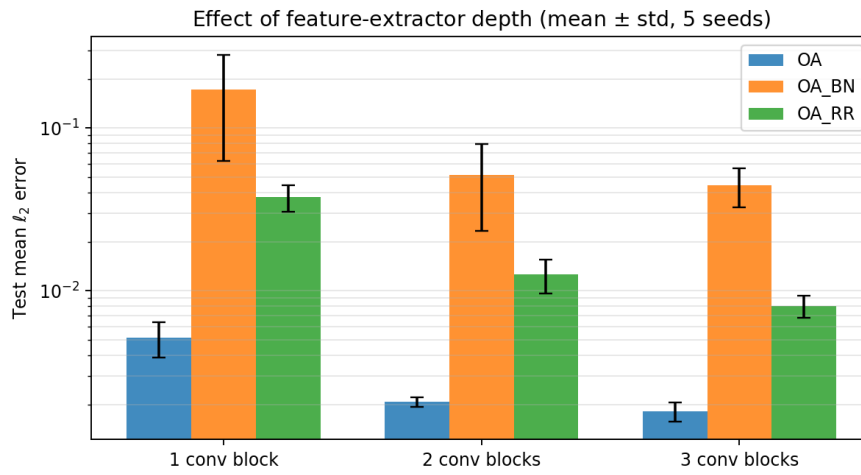


Figure 2 Test error vs. feature-extractor depth (log scale, mean $\pm$ std over 5 seeds).

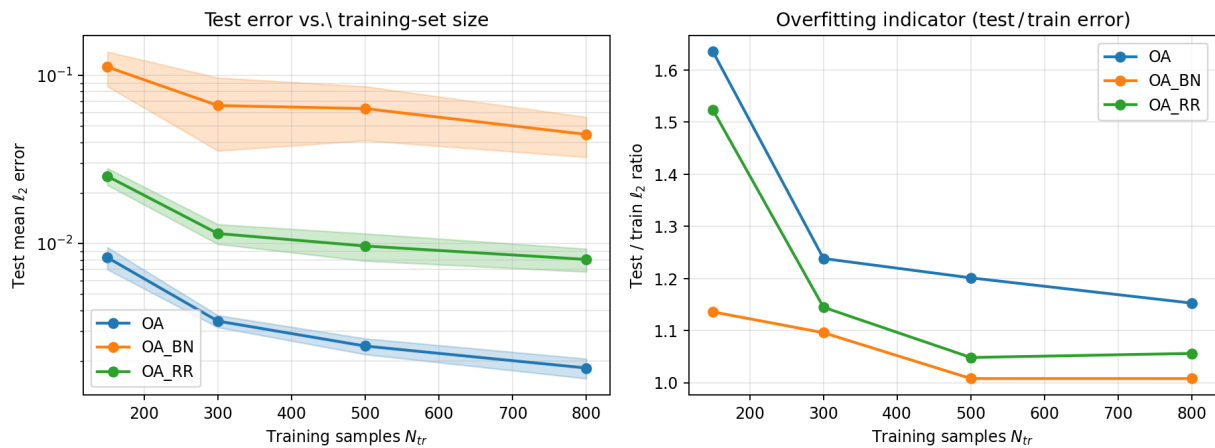


Figure 3 Data-reduction study (mean $\pm$ std over 5 seeds). Left: test error vs. training-set size (log scale). Right: test/train error ratio as an overfitting indicator.

5 Discussion

The Week-1 conclusion was right, and is now trustworthy. With seeded data generation and five seeds, the single-run Week-1 ranking ($OA < OA_RR < OA_BN$) is reproduced with non-overlapping error bars in every configuration we tested. The main change is quantitative: BN’s penalty was, if anything, *under*-estimated by the single Week-1 run.

Why BN hurts here. The literature we read this week points to the answer: BN’s benefits are optimization benefits — landscape smoothing [6], tolerance of large learning rates [7], implicit regularization [8] — that pay off in deep networks under aggressive training. Our network is shallow, the task is noiseless, and Adam at $lr\ 10^{-3}$ already optimizes it easily (OA reaches $\sim 10^{-6}$ train MSE). BN therefore contributes no optimization benefit while destroying task-relevant information: the blob’s position is encoded in *which* features are active and *how strongly*, and per-feature standardization plus a train/inference statistics mismatch distorts exactly that. Consistently, BN’s near-unity test/train ratio shows its error is a systematic distortion, not overfitting.

Why the depth ablation amplifies the gap. With three conv blocks, the layers after the bottleneck see a compact 2048-dimensional representation and the network has capacity to partially compensate for the bottleneck’s distortion. With one block, the bottleneck acts on a raw 8192-dimensional, highly spatially-correlated representation and only the small head follows it: there is less downstream capacity to undo the damage. OA_RR degrades gracefully because its rank-8 projection preserves the dominant joint structure of the batch (the optimal rank- k approximation), whereas BN rescales 8192 coordinates independently and its batch statistics become the dominant error source — also explaining its enormous seed variance at depth 1.

The regularization question is still open. The mentor’s hypothesis — that BN or RR should “bring something to the table” once the baseline overfits — could not yet be confirmed or refuted, simply because OA barely overfits even with 150 samples (test/train ratio ~ 1.7 , test error still 8×10^{-3}). The distortion introduced by either bottleneck ($\sim 10^{-2}$) is an order of magnitude larger than the overfitting gap it could possibly close. Making the task genuinely hard (noise, fewer samples than rank constraints, or harder targets) is the natural next step.

Unexpected finding. The custom SVD backward was innocent: 0 failures and 0 NaNs in all runs. The simpler `stable_svd` is still preferable — marginally better accuracy, half the seed variance, and far less code to maintain.

6 Limitations and Future Work

- **The task never reaches the regime where bottlenecks should help.** Even at 150 training samples the plain baseline barely overfits, so we could not test the regularization value of BN or RR. *Next:* add input/label noise, reduce data below 150, or move to a harder task (e.g. the autoencoder setting of the RRAE paper) where overfitting is real.
- **One rank, one batch size.** All RR results use $k = 8$ and batch size 32. *Next:* sweep the rank k and the batch size, since both directly control how much information the RR projection retains and how noisy the per-batch basis is.
- **Bottleneck position and count.** We only ever insert a single bottleneck at the flatten point. *Next:* test BN/RR after each conv block (BN’s usual placement), which may treat BN more fairly.
- **Five seeds, one dataset seed.** Five training seeds on one fixed dataset quantify training variance but not dataset variance. *Next:* also vary the data seed, and report both.
- **CPU-only budget.** All 75 runs were trained on a single CPU core, capping us at 20 epochs.

Next: longer training on GPU to check whether BN's optimization benefits appear late.

7 Conclusion

This week we turned last week's single anecdotal run into a statistically grounded comparison: we seeded the data generation, re-implemented Batch Normalization from scratch, simplified the RR layer's SVD to rely on native autograd, and evaluated everything over five seeds, two ablation axes (extractor depth and training-set size), and two SVD implementations — 75 training runs in total. The verdict is consistent everywhere: $OA < OA_RR < OA_BN$, with the SVD-based RR bottleneck about $5\times$ more accurate than BN on average and degrading far more gracefully as the network shrinks or data get scarce. The key takeaway is that on an easy, noiseless task, batch-statistics bottlenecks can only subtract — and the RR layer subtracts an order of magnitude less than BN. Whether either can *add* value must be answered in a genuinely overfitting-prone setting, which is our target for Week 3.

References

- [1] Jad Mounayer et al. “Rank Reduction Autoencoders”. In: *arXiv preprint arXiv:2405.13980* (2024).
- [2] Carl Eckart and Gale Young. “The approximation of one matrix by another of lower rank”. In: *Psychometrika* 1.3 (1936), pp. 211–218.
- [3] Sergey Ioffe and Christian Szegedy. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *International Conference on Machine Learning (ICML)*. 2015.
- [4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer Normalization”. In: *arXiv preprint arXiv:1607.06450* (2016).
- [5] Yuxin Wu and Kaiming He. “Group Normalization”. In: *European Conference on Computer Vision (ECCV)*. 2018.
- [6] Shibani Santurkar et al. “How Does Batch Normalization Help Optimization?” In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.
- [7] Nils Bjorck et al. “Understanding Batch Normalization”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.
- [8] Ping Luo et al. “Towards Understanding Regularization in Batch Normalization”. In: *International Conference on Learning Representations (ICLR)*. 2019.
- [9] Catalin Ionescu, Orestis Vantzos, and Cristian Sminchisescu. “Matrix Backpropagation for Deep Networks with Structured Layers”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2015.
- [10] Diederik P Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [11] Adam Paszke, Sam Gross, Francisco Massa, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019.